

Учебный набор «События онлайн-сервиса»

Набор полностью синтетический: строки создаёт `generate_dataset.py`, реальных клиентов и персональных данных в источнике нет. Он предназначен для демонстрации типов, проверок качества, хранения и запросов, а не для продуктовых выводов.

Начните с генерации малого учебного CSV на 200000 строк и воспроизведите путь CSV → Parquet → Iceberg → SQL по [условию ДЗ](#). Размеры и время измеряйте в своём запуске: компактность файла не гарантирует ускорение запроса.

Получение и воспроизводимость

Команда из каталога `infra/`:

```
python3 generate_dataset.py --rows 200000 --out data/events_demo.csv
```

Это также значения по умолчанию. Допускается только положительное число строк. Файл — UTF-8 CSV с заголовком из 21 поля; 200000 событий означают 200001 строку в этом формате генератора с учётом заголовка. Фактический размер выводится после генерации; заранее фиксированный объём не обещается. Для повторения пути в ДЗ не требуется увеличивать учебный набор до двух миллионов строк.

Seed по умолчанию — `20260910`. Одинаковые seed, число строк и реализация генератора воспроизводят строки. При том же seed короткий набор является префиксом длинного; новый запуск начинает те же `event_id` заново. Это **не новая поставка** для `append`. Для демонстрации двух записей делим один набор по дате. При переносе между версиями Python полагаться на совпадение файла без проверки не следует.

Период генератора: **2026-09-01 00:00:00 включительно — 2026-11-30 00:00:00 исключительно** (90 дней, последняя возможная дата — 29 ноября). Даты учебные. Строка `timestamp` не содержит смещения; договорённость набора — **UTC**. Скрипты 02 и 03 явно задают `spark.sql.session.timeZone=UTC`.

Словарь 21 поля

CSV не хранит типы. Ниже — явные типы чтения из `02_csv_to_parquet.py`. Пустые финансовые значения читаются как `null`, а не ноль.

Поле	Тип Spark SQL	Смысл и ограничения генератора
<code>event_id</code>	STRING	Seed и порядковый номер строки через дефис; уникален внутри одного файла.
<code>user_id</code>	INT	Случайный условный идентификатор 100000–399999.
<code>session_id</code>	STRING	Случайная строка <code>s</code> и номер из пула 1000000; не связана с пользователем.
<code>event_ts</code>	TIMESTAMP	Случайная секунда указанного периода, по договорённости UTC.
<code>event_date</code>	DATE	Дата того же <code>event_ts</code> ; отдельное поле для Parquet-партиций.
<code>event_type</code>	STRING	Один из восьми типов, перечисленных ниже.

Поле	Тип Spark SQL	Смысл и ограничения генератора
platform	STRING	web, android, ios.
device_type	STRING	Для web: desktop/smartphone/tablet; для android/ios: smartphone/tablet.
os_version	STRING	Условная версия 13–18, не реальная модель ОС.
app_version	STRING	Одна из шести условных версий от 9.1.0 до 10.1.0.
region	STRING	Один из 18 регионов списка генератора.
city	STRING	Город, закреплённый за выбранным регионом в коде.
referrer	STRING	Один из 10 условных источников перехода, включая direct.
category_id	INT	1000 + 7 × индекс категории, индекс 0–19.
category_name	STRING	Соответствующая категория; назначается всем типам событий.
price_rub	DECIMAL(12,2)	У purchase/refund: 290.00–299989.99; у остальных null.
quantity	INT	У purchase/refund: 1–4; у остальных null.
currency	STRING	Всегда RUB, в том числе у событий без цены.
payment_method	STRING	У purchase/refund: card/sbp/wallet/cash_on_delivery/installment; у остальных null.
duration_ms	INT	Независимая случайная величина 100–599999 мс; не длительность сессии.
ab_group	STRING	Независимый выбор A/B для каждого события, не закреплён за пользователем.

Типы событий: `page_view`, `item_view`, `click`, `search`, `add_to_cart`, `remove_from_cart`, `purchase`, `refund`. Их веса в коде — соответственно 30, 25, 20, 10, 6, 2, 6, 1. Это параметры случайного выбора, а не измеренные проценты существующего файла.

Что можно и нельзя объяснять этим набором

– Можно считать события по категориям и датам, демонстрировать null, явные типы, агрегации, сохранность строк, партиции и две записи в Iceberg.

– Воронка не моделируется: пользователь, сессия, тип и время события выбираются независимо. Отношение `purchase/page_view` — отношение числа событий, не пользовательская конверсия. Порядок строк не является порядком времени.

– `price_rub` — условное денежное поле. Генератор не определяет, цена ли это единицы или сумма заказа; заказов и товаров с идентификаторами нет. Возврат не связан с покупкой и имеет положительное значение. Нельзя называть сумму или произведение на `quantity` выручкой, рассчитывать прибыль либо `net revenue`. В демо используем «среднее значение `price_rub` у событий `purchase`».

- A/B-группа меняется от события к событию. Набор не подходит для выводов об эффекте эксперимента, удержании и поведении реальных сессий.
- Условные версии ОС/приложения, категории и `duration_ms` не описывают реальную эксплуатацию продукта. Связь `region/city` и `category_id/category_name` задана явно, но остальные правдоподобные связи не следует достраивать.
- Генератор специально не вносит грязные строки и дубликаты. Проверки должны подтверждать контракт; обещать, что они найдут ошибку, нельзя.

Контрольные точки демонстрации

1. `01_upload_csv.py` берёт `/data/events_demo.csv` и записывает объект `s3://raw/events/2026/events.csv`. Ключ объекта не является таблицей или встроенным фильтром по дате.
2. `02_csv_to_parquet.py` читает все 21 поле по схеме, останавливается на ошибке CSV, проверяет обязательные значения, уникальность `event_id`, допустимость типов событий, согласованность дат и финансовых полей. Для шести типов без финансового события null цены, количества и способа оплаты **корректен**. Такие строки сохраняются; фильтра `price_rub > 0` для всей таблицы нет.
3. Все события записываются в `s3a://datalake/warehouse/events_parquet` с `snappy` и партициями по `event_date`. Количество строк после чтения сверяется с CSV. 90 дней на небольшом наборе могут дать множество маленьких файлов: это иллюстрация организации, а не рекомендация оптимального размера партиций.
4. Для обоих форматов выполняется один SQL-запрос:

```
SELECT count(*) AS purchase_events,  
       avg(price_rub) AS mean_purchase_price  
FROM events  
WHERE event_type = 'purchase'  
      AND event_date >= DATE '2026-09-01'  
      AND event_date < DATE '2026-10-01';
```

В скрипте меняется только имя временного представления. По три измерения каждого формата, первый обозначен отдельно, порядок форматов чередуется. Результаты всех измерений сверяются, включая `Decimal`. Первый замер не называется холодным: проверки и запись уже читали данные. Кэши, прогрев JVM, партиции и накладные расходы влияют на время; ускорение не гарантируется.

5. `03_iceberg.py` до записи проверяет уникальные идентификаторы, согласованные даты и две непустые порции. Первая — `event_date < 2026-10-01`, вторая — `event_date >= 2026-10-01`. CTAS первой порции пишет новые Iceberg-файлы; INSERT добавляет вторую порцию. Таблица — `lakehouse.dwh.events`, партиции — `days(event_ts)`. Показываются число строк и новые snapshot IDs после каждой записи; финальное число строк и уникальность ID сверяются с Parquet.

6. Повторный запуск 03 заменяет содержимое только этой учебной таблицы через `CREATE OR REPLACE` и затем снова добавляет вторую порцию. Он предназначен для повторения демо без накопления дублей. История может сохранить прежние snapshots: показываем **две новые записи**, а не обещаем ровно два снимка за всё время. Не выполнять отдельный INSERT повторно поверх готовой таблицы.

7. Общее число строк растёт после второй записи, но запрос за сентябрь остаётся тем же: вторая порция относится к октябрю–ноябрю. Trino читает таблицу через общий JDBC-каталог; его LEFT JOIN со справочником `memory.default.region_groups` сохраняет регионы без соответствия под именем «Не сопоставлен». Справочник намеренно содержит 5 из 18 регионов, а результат JOIN также ограничен сентябрём.

Студенты наблюдают и обсуждают эти проверки на семинаре. Подготовка стенда и самостоятельное выполнение команд относятся к домашней работе. Перед повторением проверьте текущее состояние сервисов по инструкции своей ОС. Логи своих запусков сохраните в [evidence/](#) каталога сдачи; чужие результаты не заменяют ваши проверки.

Показ набора на семинаре

Презентация семинара содержит вводную и завершение. Содержимое CSV показывается в редакторе, ключи и размеры объектов — в MinIO. Установленная MinIO Console не поддерживает preview CSV, JSON и Parquet; скачанный JSON метаданных открывается в редакторе. Строки Iceberg и результаты SQL показываются в DataGrip, подключённом к Trino. DataGrip сам не заменяет движок чтения файлов.

Последовательность SQL — [trino/scripts/seminar_demo.sql](#): в основном запросе AVG приводится к DECIMAL с шестью знаками, чтобы сравнить его со Spark; в результате JOIN среднее округлено до двух знаков. Файл содержит контроль уникальности справочника и количества событий до и после JOIN.